CONTRACT WOLF

Security Assessment

DARKPOOL

Verified on October 16, 2026

SUMMARY

Project	CHAIN	METHODOLOGY
DARKPOOL	Solana	Manual & Automatic Analysis

FILES	DELIVERY	TYPE
Repository	October 16, 2026	Standard Audit



9	0	2	0	4	3	9
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

🟥 0 Critical	An exposure that can affect the contract functions in several events that can risk and disrupt the contract
🟧 2 Major	An opening & exposure to manipulate the contract in an unwanted manner
🟨 0 Medium	An opening that could affect the outcome in executing the contract in a specific situation
⬛ 4 Minor	An opening but doesn't have an impact on the functionality of the contract
🟦 3 Informational	An opening that consists information but will not risk or affect the contract
🟩 9 Resolved	ContractWolf's findings has been acknowledged & resolved by the project

STATUS ✔️**AUDIT PASSED**

TABLE OF CONTENTS | DARKPOOL

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details

Findings

Issues
Rust Attacks
Fixes & Recommendation
Audit Comments

DISCLAIMER | DARKPOOL

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | DARKPOOL

DARKPOOL team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **DARKPOOL**.

AUDITING APPROACH | DARKPOOL

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | DARKPOOL



DARKPOOL is a shielded pool for private trading. Users shield coins into private notes, swap privately through Jupiter (Solana) or Uniswap V3/V4 (Robinhood Chain), send privately and unshield to any address.

Token Name	Symbol	Decimal	Total Supply	Chain
DARKPOOL	DARKPOOL	6	-	Solana

SOURCE

Source	*GitHub Repository (darkpoolsol/darkpool-audit, commit 71b1a76) and Solana Program Gj6tgwJ8bKMUUoSxWrk57APLZbk6gJ17yFnQy8HA2qoK*

FINDINGS | DARKPOOL



9	0	2	0	4	3	9
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

This report has been prepared to state the issues and vulnerabilities for DARKPOOL through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
SOL-SYNC-DOS	One stray lamport permanently blocks SOL shielding	solana/programs/darkpool/src/lib.rs, L: 192-205	Major	Resolved
RCW-011	listable() admits mints that keep a freeze authority	solana/programs/darkpool/src/funds.rs, L: 348-371	Major	Resolved
ROUTER-LAMPORTS	The router's lamport balance is not restored after a route	solana/programs/darkpool/src/lib.rs, L: 584-589	Minor	Resolved
ROUTE-FLAGS	The route digest omits each account's signer and writable flags	solana/programs/darkpool/src/crypto.rs, L: 246-255	Minor	Resolved
PROGRAM-ID	declare_id! does not match the deployed program	solana/programs/darkpool/src/lib.rs, L: 48	Minor	Resolved
PRICE-ORACLE	The buyback guard takes its price and its minimum from one endpoint	src/darkpool/keeper.mjs, L: 225-232	Minor	Resolved
MARKER-TREE	Spent markers bind a note's position in its tree, not which tree	circuits/transact.circom, L: 76	Informational	Resolved
VAULT-ROUNDING	A gift can coarsen the next vault deposit by up to one fine share	solana/programs/darkpool/src/lib.rs, L: 142-156	Informational	Resolved

VAULT-LEGACY	The vault's legacy proof fallback doubles the cost of a failed vault spend	solana/programs/darkpool/src/lib.rs, L: 420-426	Informational	• Resolved

RUST ATTACKS | DARKPOOL

Rust Smart Contract Weakness Classification and Test Cases

ID	Name	Description	Status
RCW-001	Reentrancy	A malicious contract calls back into the calling contract before the first invocation of the function is finished.	Passed
RCW-002	Undefined behavior	The Rust reference contains a non-exhaustive list of behaviors considered undefined in Rust.	Passed
RCW-003	Integer overflow & underflow	Overflow or underflow of mathematical operations inside the Rust smart contract.	Passed
RCW-004	Out of bounds read/write	The contract or function reads data past the end, or before the beginning, of the intended buffer.	Passed
RCW-005	Memory corruption	Wrong usage of the memory model throughout the contract or within its functions.	Passed
RCW-006	Typographical error	Unintended error in contract and function names, code, and arithmetic inputs.	Passed
RCW-007	Hash collisions with multiple arguments	If used incorrectly, triggers a hash collision while calling a function within a function.	Passed
RCW-008	Account confusion	Failure to verify account ownership or types, leading to unauthorized access.	Passed
RCW-009	Insufficient rent exemption	Accounts not properly funded for rent exemption, risking purge.	Passed
RCW-010	Unverified PDA derivations	Not validating Program Derived Address (PDA) seeds or bump.	Passed
RCW-011	Missing authority checks	Unauthorized token minting via unverified authorities, and improper freeze authorities.	Passed
RCW-012	Arithmetic precision loss	Truncation errors in financial calculations and default round-offs (e.g. 0.999).	Passed
RCW-013	Compute unit exhaustion	Unbounded loops causing transaction failures.	Passed
RCW-014	Reward distribution flaws	Inaccurate reward pro-rata calculations.	Passed
RCW-015	Slot-based race conditions	Front-running via transaction ordering.	Passed
RCW-016	Uncontrolled allocations	Memory allocation without size limits.	Passed
RCW-017	Erroneous computation	Incorrect or unintended math calculation.	Passed
RCW-018	Function manipulation	Manipulation attack of time-difference values to increase rewards.	Passed
RCW-019	Denial of service	DDoS attack using pending transactions.	Passed
RCW-020	High verbosity logging	The product does not properly control the allocation and maintenance of a limited resource.	Passed
RCW-021	Incorrect SPL token account delegate expiration	Verifies proper revocation of delegated token permissions after expiration	Passed

ID	Name	Description	Status
RCW-022	Unverified vault authority in token swaps	Checks validation of swap vault ownership, to prevent malicious actors draining liquidity pools via unauthorized access.	Passed
RCW-023	Solana BPF loader address spoofing	Tests detection of forged BPF program IDs, to block execution of malicious smart contracts masquerading as trusted programs.	Passed
RCW-024	Missing signer checks	Instructions that change state or move funds without confirming the required account actually signed the transaction.	Passed
RCW-025	Arbitrary cross-program invocation	Calling a program ID supplied by the caller instead of a verified one, handing control to attacker-written code.	Passed
RCW-026	Duplicate mutable accounts	The same account passed twice under different names, letting one instruction overwrite its own state or bypass balance checks.	Passed
RCW-027	Account data reinitialization	Initialization that can be run again on a live account, resetting authorities or balances to attacker-chosen values.	Passed
RCW-028	Account closing and revival	Closed accounts whose lamports are drained but data is not zeroed, so they can be revived within the same transaction.	Passed
RCW-029	Type cosplay	Account data deserialized without a discriminator check, so one account type is accepted in place of another.	Passed
RCW-030	Sysvar spoofing	Sysvar accounts such as Clock or Instructions read from caller-supplied accounts without verifying their addresses.	Passed
RCW-031	Bump seed canonicalization	PDAs accepted with non-canonical bump seeds, producing multiple valid addresses for one logical account.	Passed
RCW-032	Seed collisions	PDA seed schemes where different inputs concatenate to the same address, merging separate users' state.	Passed
RCW-033	Unsafe Rust blocks	Use of unsafe code, raw pointers and transmute that bypass the compiler's memory and type guarantees.	Passed
RCW-034	Panics and unchecked unwrap	unwrap, expect, indexing and division that abort the program on attacker-controlled input and block critical paths.	Passed
RCW-035	Unsafe numeric casts	Truncating or sign-changing casts between integer widths that silently alter amounts.	Passed
RCW-036	Token-2022 extension handling	Transfer fees, hooks, and permanent delegate extensions that break assumptions about transferred amounts and custody.	Passed
RCW-037	Stale account data after CPI	Reading a balance or state value cached before a cross-program invocation that has since changed it.	Passed
RCW-038	Oracle price manipulation	Reliance on a single spot price or stale, low-confidence oracle feed that can be moved or outdated.	Passed
RCW-039	Upgrade authority risk	Programs left upgradeable under a single key, or with the authority misconfigured, allowing code replacement.	Passed
RCW-040	Rounding direction errors	Rounding that favors the user instead of the protocol on deposits, withdrawals and fees, enabling dust extraction.	Passed
RCW-041	Instruction introspection abuse	Checks on sibling instructions in the transaction that can be satisfied by crafted ordering or padding.	Passed
RCW-042	Vulnerable dependencies	Crates in Cargo.lock with known advisories, unpinned versions, or unaudited forks pulled into the build.	Passed

FIXES & RECOMMENDATION

SOL-SYNC-DOS | One stray lamport permanently blocks SOL shielding

On the `WSOL` path `shield` transfers lamports into the reserve, calls `sync_native`, then requires the reserve's recorded `amount` to have risen by exactly `amount`. `sync_native` credits every lamport the account holds above rent, including lamports anyone sent straight to the reserve address. One unsolicited lamport makes the delta larger than `amount`, so every later SOL shield reverts with `TransferMismatch`, and nothing else in the program calls `sync_native` to absorb the surplus.

```
if a.entry.mint == WSOL {
    send_lamports(&a.depositor.to_account_info(), &a.reserve.to_account_info(),
                  &a.system_program.to_account_info(), amount)?;
    ti::sync_native(CpiContext::new(a.token_program.to_account_info(),
        ti::SyncNative { account: a.reserve.to_account_info() }))?;
} else {
    // transfer_checked ...
}
a.reserve.reload()?;
require!(a.reserve.amount.checked_sub(before) == Some(amount), DpError::TransferMismatch);
```

Recommendation

Require the reserve to have risen by at least `amount` on the SOL path and keep exact equality for `transfer_checked`, so donated lamports become unclaimed surplus instead of a permanent revert. Surplus is safe because `covered` only checks a lower bound.

```
let gained = a.reserve.amount.checked_sub(before).ok_or(DpError::TransferMismatch)?;
if a.entry.mint == WSOL {
    // sync_native also credits lamports sent straight to the reserve: never
    // fewer than `amount`, and anything over stays as unclaimed surplus.
    require!(gained >= amount, DpError::TransferMismatch);
} else {
    require!(gained == amount, DpError::TransferMismatch);
}
```

RCW-011 | listable() admits mints that keep a freeze authority

`listable` returns `Ok` for every mint owned by the classic SPL token program without reading the mint at all, and for a Token-2022 mint whose data is 82 bytes or shorter. Neither path looks at `freeze_authority`. The holder of a listed mint's freeze authority can freeze the program's reserve token account, after which `transact`, `swap` and `collect_fees` all fail for that coin and every note of it is unspendable for good. This contradicts invariant I14.

```
pub fn listable(mint: &AccountInfo) -> Result<()> {
    if *mint.owner == anchor_spl::token::ID {
        return Ok(());
    }
    require_keys_eq!(*mint.owner, anchor_spl::token_2022::ID, DpError::UnsupportedCoin);
    let data = mint.try_borrow_data()?;
    if data.len() <= 82 {
        return Ok(());
    }
    // ... extension walk ...
}
```

Recommendation

Read the mint's base layout on both paths and refuse any mint whose `freeze_authority` option is set, before the extension walk. The field sits at offset 46 of the SPL mint layout, in the same place for Token-2022.

```
/// The SPL mint layout: mint_authority COption (0..36), supply (36..44),
/// decimals (44), is_initialized (45), freeze_authority COption (46..82).
fn no_freeze_authority(data: &[u8]) -> Result<()> {
    require!(data.len() >= 82 && data[46..50] == [0u8; 4], DpError::UnsupportedCoin);
    Ok(())
}

pub fn listable(mint: &AccountInfo) -> Result<()> {
    let data = mint.try_borrow_data()?;
    no_freeze_authority(&data)?;
    if *mint.owner == anchor_spl::token::ID {
        return Ok(());
    }
    require_keys_eq!(*mint.owner, anchor_spl::token_2022::ID, DpError::UnsupportedCoin);
    if data.len() <= 82 {
        return Ok(());
    }
    // ... extension walk unchanged ...
    Ok(())
}
```

ROUTER-LAMPORTS | The router's lamport balance is not restored after a route

`run_route` invokes the routing program with the router address's signature, then checks that the router is still a system account with no data and that both of its token accounts keep their owner, delegate and close authority. It never compares the router's lamports before and after. An approved routing program therefore holds a signature that can move the router's SOL float out, so invariant I11 holds for the router's ownership and data but not for its balance.

```
invoke_signed(&Instruction { program_id: *program.key, accounts: metas, data:
route.to_vec() },
              &infos, &[router_seeds])?;
require!(router.owner == &anchor_lang::system_program::ID && router.data_is_empty(),
         DpError::RouteRejected);
router_account_intact(&in_info, &router.key())?;
router_account_intact(&out_info, &router.key())?;
```

Recommendation

Record the router's lamports before the invoke and require them to be unchanged afterwards, so the whole account, not only its owner and data, comes back as it was.

```
let lamports_before = router.lamports();
invoke_signed(&Instruction { program_id: *program.key, accounts: metas, data:
route.to_vec() },
              &infos, &[router_seeds])?;
require!(router.owner == &anchor_lang::system_program::ID && router.data_is_empty()
         && router.lamports() == lamports_before, DpError::RouteRejected);
```

ROUTE-FLAGS | The route digest omits each account's signer and writable flags

`route_digest` hashes the routing program, the instruction data and the account addresses, and nothing else. `run_route` then builds the metas from whatever flags the relayer put on those accounts in its own transaction. A relayer can therefore change a route's `is_signer` and `is_writable` flags without changing `extHash`, so invariant I5 binds the route's account list but not the shape in which it is presented.

```
pub fn route_digest(program: &Pubkey, data: &[u8], accounts: &[AccountInfo]) -> [u8; 32] {
    // ...
    for a in accounts {
        buf.extend_from_slice(a.key.as_ref());
    }
    keccak::hash(&buf).0
}
```

Recommendation

Hash one flag byte per account alongside its address so the digest covers the metas the route actually receives. The proven `min_out` limits what flag changes can achieve today, but the binding should not depend on that.

```
for a in accounts {
    buf.extend_from_slice(a.key.as_ref());
    // the flags the route is invoked with, not only which accounts it gets
    buf.push((a.is_signer as u8) | ((a.is_writable as u8) << 1));
}
```

PROGRAM-ID | declare_id! does not match the deployed program

The audited source declares `BDvCkXwcfZPhBhY3Dg2jLX15Vhvu12U2vHikcwyJUjcp`, while the live program is `Gj6tgwJ8bKMUUoSxWrk57APLZbk6gJ17yFnQy8HA2qoK`. The id is compiled into every PDA derivation and into both `extHash` domains, so this source cannot reproduce the deployed ELF and no proof built against it would verify on mainnet. The deployed bytecode could not be matched to the audited code from this package.

```
declare_id!("BDvCkXwcfZPhBhY3Dg2jLX15Vhvu12U2vHikcwyJUjcp");
```

Recommendation

Set `declare_id!` to the deployed address, rebuild with `cargo-build-sbf`, and publish the resulting ELF hash next to the deployed program's so anyone can confirm the two agree.

```
declare_id!("Gj6tgwJ8bKMUUoSxWrk57APLZbk6gJ17yFnQy8HA2qoK");
```

PRICE-ORACLE | The buyback guard takes its price and its minimum from one endpoint

`observe` builds the reference median from Jupiter quotes, `buyback` reads its spot from the newest of those same quotes, derives the floor from it, and then sends Jupiter's own quote, so the on-chain minimum is Jupiter's `otherAmountThreshold`. Every number the guard compares comes from one off-chain endpoint, so a wrong or spoofed response moves the reference, the floor and the enforced minimum together and the guard still passes.

```
const spot = this.observations().at(-1)?.spot;
if (!(spot > 0) || spot < average * (1 - this.guardBps / 10000)) { /* wait */ return; }
for (let i = 0; i < 8 && amount >= this.minBuy; i++, amount /= 2n) {
  const floor = BigInt(Math.ceil((Number(amount) / 1e9) * spot * (1 - this.impactBps /
10000)));
  const q = await this.quote(mint, amount, floor);
  if (q && q.out >= floor && q.min >= floor) { quote = q; break; }
}
```

Recommendation

Derive the floor from a second price read directly from the chain, such as the pump.fun curve or the pool reserves, and refuse to buy when the two sources disagree by more than `guardBps`.

```
// an on-chain second opinion, read without going through the quote API
const onchain = await this.onchainSpot(mint);      // tokens per SOL, from reserves
if (!(onchain > 0)) { this.log('keeper buyback: no on-chain price; waiting'); return; }
if (Math.abs(spot - onchain) / onchain > this.guardBps / 10000) {
  this.log('keeper buyback: quote and on-chain price disagree; waiting');
  return;
}
const reference = Math.min(spot, onchain);         // the floor uses the worse of the two
```

MARKER-TREE | Spent markers bind a note's position in its tree, not which tree

The marker is `H("dp/spent", nk, C, inLeaf)` with `inLeaf` range checked to 20 bits, so it names a position inside a tree and not a global leaf number. Two identical commitments at the same position in two different trees share one marker. `hide` is public for every note the program makes, so such a copy can be built, but it belongs to the original address: only one of the pair is ever spendable and the copier loses their own deposit.

```
marker[i] = Poseidon(4);
marker[i].inputs[0] <== TAG_SPENT();
marker[i].inputs[1] <== keys.nk;
marker[i].inputs[2] <== inNote[i].commitment;
marker[i].inputs[3] <== inLeaf[i];
nullifiers[i] <== marker[i].out * (1 - unused[i].out);
```

Recommendation

Add the tree number as a fifth marker input and check it against the root the proof uses, so markers become injective over notes. No one is harmed today, but the collision leaves no margin for a later change to the tree layout.

```
signal input inTree[nIn];            // which tree the note sits in
// ...
marker[i] = Poseidon(5);
marker[i].inputs[0] <== TAG_SPENT();
marker[i].inputs[1] <== keys.nk;
marker[i].inputs[2] <== inNote[i].commitment;
marker[i].inputs[3] <== inLeaf[i];
marker[i].inputs[4] <== inTree[i];
```

VAULT-ROUNDING | A gift can coarsen the next vault deposit by up to one fine share

With the vault at version 2 and no holders, `donate` accepts up to `MAX_UNITS_PER_SHARE` times `VIRTUAL_SHARES` minus one, which puts one fine share at up to 1,000,000 token units. A later `shield` then rounds its shares down by as much as 999,999 units, about one whole $DARKPOOL at six decimals. The donor holds no shares, so none of it reaches them: the loss is a griefing cost, not an extraction.

```
a.entry.backing = a.entry.backing.checked_add(amount).ok_or(DpError::BadAmount)?;
if a.entry.vault == 2 {
    require!(a.entry.backing as u128 + 1
             <= MAX_UNITS_PER_SHARE * (a.entry.shielded as u128 + VIRTUAL_SHARES),
             DpError::GiftTooLarge);
}
```

Recommendation

Lower `MAX_UNITS_PER_SHARE` so one fine share stays worth a few units, which keeps the worst rounding below a visible amount while still letting the vault grow with real deposits behind it.

```
/// A gift may never make one fine share worth more than this many token units.
/// At 6 decimals this caps any deposit's rounding loss at 0.000099 $DARKPOOL.
const MAX_UNITS_PER_SHARE: u128 = 100;
```

VAULT-LEGACY | The vault's legacy proof fallback doubles the cost of a failed vault spend

`check_spend_coin` retries the whole Groth16 check against `coin_id(mint)` whenever the first check fails on a version 2 vault entry. On a fresh deployment `list_coin` sets `vault` to 2 and `coin` to `coin_id_v2(mint)` straight away, and the circuit forces inputs and outputs to share one coin, so no note can ever carry `coin_id(mint)`. The branch is unreachable and only doubles the worst-case compute of a failed vault spend.

```
fn check_spend_coin(state: &State, h: &SpendHead, entry: &TokenEntry, ext: Fe) ->
Result<bool> {
    match check_spend(state, h, entry.coin, ext) {
        Ok(()) => Ok(false),
        Err(e) if entry.vault == 2 && is_invalid_proof(&e) =>
            check_spend(state, h, coin_id(&entry.mint), ext).map(|_| true),
        Err(e) => Err(e),
    }
}
```

Recommendation

Gate the fallback on a marker that `upgrade_vault` sets, so only an entry that really carried version 1 notes pays for the second verification and a freshly listed vault coin never does.

```
// `vault == 3`: migrated from version 1, so pre-change notes may still exist.
Err(e) if entry.vault == 3 && is_invalid_proof(&e) =>
    check_spend(state, h, coin_id(&entry.mint), ext).map(|_| true),
```

AUDIT COMMENTS | DARKPOOL

Smart Contract audit comment for a non-technical perspective

- There is no pause, no freeze, no blocklist and no fee switch. The 0.5% swap fee and the 80/20 split are constants that only an upgrade can change.
- The admin cannot touch a holder's notes. Spending requires the holder's spend key inside the proof, and the admin's only powers are `configure`, `set_vault` once, `set_router` and `upgrade_vault`.
- The vault coin can be named only once, only by the upgrade authority, and only before that coin is listed, so no note of it can exist in plain amounts first.
- Anyone can relay. Each action binds its relayer and the relayer's fee into the proof, so a copied proof is worthless to anyone else and no single operator has to be trusted.
- Every reserve is checked to cover its notes plus uncollected fees after each movement, and swap amounts are measured from real balance changes rather than taken from the route's own report.
- Anyone can call `collect_fees`, and it can only pay into the fee wallet's and the builder's associated token accounts, so the destination of the fee stream is fixed.



CONTRACTWOLF

Blockchain Security - Smart Contract Audits